CS 7180: Production Readiness & Emerging AI Engineering

Northeastern University

John Alexis Guerra Gomez
jguerra at northeastern.edu
Class: **johnguerra.co/classes/aiCoding_spring_2026**
Slides: **johnguerra.co/lectures/ai_assisted_coding**

What We'll Cover Today

1. Where We Are -- Week 14 checkpoint
2. Production Deployment
3. Monitoring & Observability
4. Performance Optimization with AI
5. Cost Optimization
6. RAG & Vector Databases
7. AI Code Review at Scale
8. The Future of AI Engineering
9. Demo Preparation
10. Meta-Reflection & Course Synthesis

Where We Are

> *Week 14 -- Final week of lectures*

Recap: Week 13

Agent Architectures & Agent SDK

- The 6 agent patterns: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, autonomous
- Claude Agent SDK: `query()`, hooks, sessions
- Multi-agent coordination and safety

You can now build agents. This week: ship them to production.

This Week: Production & Beyond

Session 1: Get your P3 production-ready

- Deployment pipelines, monitoring, performance and cost optimization

Session 2: Prepare for Demo Day

- RAG & vector databases, AI code review at scale, future trends
- Demo rehearsal and meta-reflection

Production Deployment

> *From localhost to production*

Why Deployment Matters

Your P3 is not done until it's deployed.

- A project that only runs on localhost is a homework exercise
- A deployed project is a portfolio piece
- Employers check live URLs

The gap between "it works on my machine" and "it works in production" is where engineering happens.

Vercel for Next.js

Why Vercel? Built by the Next.js team. Zero-config, automatic HTTPS, CDN, edge functions, generous free tier.

```
npm i -g vercel
vercel login
vercel          # deploys to preview
vercel --prod   # deploys to production
```

Every PR gets its own preview URL automatically:

```
PR #42: add-login-page
  -> https://myapp-pr-42.vercel.app
```

Reviewers test the PR live before merging. No branch pulling needed.

Environment Variable Management

Never commit secrets to git.

Environment	Where secrets live
Local dev	`.env.local` (gitignored)
Preview	Vercel dashboard > Settings > Env Vars
Production	Vercel dashboard > Settings > Env Vars

```
# .env.local (gitignored)
DATABASE_URL=postgres://localhost:5432/myapp_dev

# Vercel Production (configured in dashboard)
DATABASE_URL=postgres://prod-server:5432/myapp
```

Production Deployment Gates

Never deploy directly. Let the pipeline decide.

```
main branch push -> CI/CD runs -> All pass? -> Deploy
                    Fail?     -> Block + notify
```

The rule: If CI fails, production doesn't update. Period.

The Full CI/CD Pipeline

> *Every step between `git push` and production*

Pipeline Stages

Stage	Tool	What it catches
1. Lint	ESLint, Prettier	Style violations, unused imports
2. Typecheck	`tsc --noEmit`	Type errors tests might miss
3. Unit Tests	Jest / Vitest	Logic bugs in isolated functions
4. Integration	API route tests	API contract violations
5. E2E Tests	Playwright	Broken user workflows
6. Security Scan	`npm audit`, OWASP	Known vulnerable dependencies
7. AI PR Review	Claude Code	Architectural issues, edge cases
8. Preview Deploy	Vercel preview URL	Build failures, runtime errors
9. Prod Deploy	Vercel production	Final destination

Catching bugs earlier is cheaper. Lint is free. Production bugs cost reputation.

GitHub Actions Workflow

```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npx playwright test
      - run: npm audit --audit-level=high
```

Each step must pass before the next runs.

Monitoring & Observability

> *Knowing what's happening in production*

The Three Pillars

Pillar	What it tells you	Tool
Logs	What happened	Structured logging (JSON)
Metrics	How much / how fast	Uptime monitors
Errors	What went wrong	Sentry

Without monitoring, you only find bugs when users complain.

Error Tracking with Sentry

```
import * as Sentry from "@sentry/nextjs";

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  tracesSampleRate: 0.1,
  environment: process.env.NODE_ENV,
});
```

What you get: stack traces with source maps, user context, breadcrumbs (what happened before the error), release tracking (which deploy introduced it).
With a Sentry MCP server, Claude Code can query your errors directly:

```
> "What are the top 5 errors in production this
week?"
Claude reads Sentry API -> summarizes -> suggests
fixes
```

Structured Logging

Don't: `console.log("user logged in")`

Do:

```
logger.info("user.login", {
  userId: user.id, method: "oauth", duration_ms: 142
});

logger.error("payment.failed", {
  userId: user.id, amount: 49.99, error: err.message
});
```

Structured logs are searchable, filterable, and machine-readable.

Claude Code Hooks for Production

PreToolUse: Block merges without passing CI

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash(gh pr merge*)",
      "command": "check-ci-status.sh",
      "action": "block"
    }]
  }
}
```

PostToolUse: Auto-generate release notes on tag

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Bash(git tag*)",
      "command": "generate-release-notes.sh"
    }]
  }
}
```

Performance Optimization with AI

> *Using AI to make your app faster*

Lighthouse + Claude Code

Pipe performance reports directly to Claude.

```
# Generate Lighthouse report
npx lighthouse https://myapp.vercel.app \
  --output=json --output-path=./report.json

# Ask Claude to analyze it
cat report.json | claude -p \
  "Identify the top 3 performance improvements.
   For each: issue, expected impact, exact code
changes."
```

AI reads the 5000-line JSON so you don't have to.

Common Performance Patterns

Pattern	Problem	AI-Assisted Fix
DB queries	N+1 queries, missing indexes	Claude analyzes query logs, suggests indexes
Bundle size	Large JS bundles	Claude identifies heavy imports, suggests tree-shaking
Images	Unoptimized images	Claude suggests next/image, WebP conversion
Lazy loading	Everything loads upfront	Claude identifies below-fold components for `dynamic()`
Caching	Repeated API calls	Claude suggests cache headers, stale-while-revalidate

```
// Before: imported on every page load
import HeavyChart from './HeavyChart';

// After: loaded only when needed
const HeavyChart = dynamic(
  () => import('./HeavyChart'),
  { loading: () => <Skeleton /> }
);
```

Cost Optimization

> *Managing AI costs in production workflows*

Prompt Caching: How It Works

Problem: Repeated API calls with similar prompts waste tokens.

```
First call:
  [System prompt: 2000 tokens] + [User query: 50
tokens]
  -> Full processing, result cached

Second call (same system prompt):
  [Cached: 2000 tokens] + [New query: 50 tokens]
  -> 90% cheaper, 85% faster
```

When to use: Long system prompts reused across calls, RAG with stable document context, batch processing with shared instructions.

Prompt Caching Mechanics

Aspect	Detail
Cache hit	System prompt matches exactly -> reuse
Cache miss	Any change to cached prefix -> recompute
TTL	~5 minutes of inactivity
Savings	Up to 90% cost reduction on cached tokens
Latency	Up to 85% faster on cache hit
Minimum	1024 tokens for caching to activate

Key insight: Structure prompts so the stable part comes first (system prompt, docs) and the variable part comes last (user query).

Model Routing

Not every task needs the most powerful model.

Model	Best for	Cost	Speed
Opus	Complex architecture, multi-file refactoring	$$$	Slower
Sonnet	Daily coding, PR reviews, standard features	$$	Medium
Haiku	Simple tasks, classification, batch processing	$	Fast

```
"Fix this typo in the README"          -> Haiku
($0.001)
"Implement login following auth pattern" -> Sonnet
($0.01)
"Redesign DB schema for multi-tenancy"  -> Opus
($0.10)
```

Route tasks to the cheapest model that can handle them.

Managing Costs with Claude Code

Set budget limits for automated tasks:

```
claude -p "Review this codebase for security issues"
\
  --max-budget-usd 5.00
```

`max_budget_usd` prevents runaway costs in: CI/CD pipeline reviews, batch processing, automated security scans, fan-out agent tasks.

Choosing the right model for the job:

- **Development:** Sonnet (good balance)
- **Code review CI:** Sonnet (quality matters)
- **Log classification:** Haiku (volume is high)
- **Architecture decisions:** Opus (stakes are high)

RAG & Vector Databases

> *Retrieval-augmented generation for AI-powered apps*

The Context Problem at Scale

Codebase	Files	Tokens
Small project	~50 files	~100K tokens
Medium project	~500 files	~1M tokens
Enterprise monorepo	~50,000 files	~100M tokens

You can't feed 100M tokens into a 200K context window. Solution: retrieve only what's relevant.

Claude Code's Context Management IS RAG

You've been using RAG without knowing it.

When Claude Code processes your request:

1. **Reads your CLAUDE.md** -- cached project context
2. **Uses Glob/Grep** -- just-in-time file retrieval
3. **Reads specific files** -- targeted context loading
4. **Compacts when full** -- summarizes to free space

This is retrieval-augmented generation:

- **Retrieval:** Glob, Grep, Read tools
- **Augmentation:** Adding retrieved files to context
- **Generation:** Producing code with that context

What Are Embeddings?

Embeddings turn text into numbers that capture meaning.

```
"authentication middleware" -> [0.82, -0.15, 0.43,
...]  (1536 dims)
"login security check"      -> [0.79, -0.12, 0.41,
...]  (similar!)
"database migration"        -> [0.11, 0.67, -0.33,
...]  (different)
```

Key insight: Similar meaning = nearby vectors. This enables **semantic search** -- finding relevant content even when keywords don't match.

Embedding Model	Dimensions	Best For
OpenAI text-embedding-3-small	1536	General purpose, good quality/cost
Voyage Code 3	1024	Code-specific, understands syntax

Embedding Model	Dimensions	Best For
Nomic Embed	768	Open source, self-hostable

Vector Databases

Where embeddings live so you can search them fast.

Database	Type	Best For
pgvector	Postgres extension	Already using Postgres, simple setup
Chroma	Lightweight, local	Prototyping, small projects
Pinecone	Managed cloud	Production scale, zero ops
Weaviate	Self-hosted or cloud	Hybrid search (vector + keyword)

```sql
-- pgvector: add vector column to existing table
ALTER TABLE docs ADD COLUMN embedding vector(1536);

-- Find 5 most similar documents
SELECT content, 1 - (embedding <=> query_vec) AS
similarity
FROM docs ORDER BY embedding <=> query_vec LIMIT 5;
```

pgvector is the easiest starting point -- no new infrastructure, just a Postgres extension.

Chunking Strategies

How you split your data determines retrieval quality.

Strategy	How it works	Best for
Fixed-size	Split every N tokens with overlap	Documents, articles
Semantic	Split at paragraph/section boundaries	Structured docs, markdown
AST-based	Split at function/class boundaries	Source code
Sliding window	Overlapping chunks for context continuity	Long narratives

For code: AST-based chunking preserves function boundaries. A function split across two chunks loses meaning.

```
// Good: one chunk per function
chunk_1: function authenticate(user, pass) { ... }
chunk_2: function validateToken(token) { ... }

// Bad: split mid-function
chunk_1: function authenticate(user, pass) { if (user
chunk_2: === null) { return false; } ... }
```

Building a RAG Pipeline

Common pitfalls: chunks too large (noisy), chunks too small (no context), no overlap (missed boundaries), stale index (data changed but embeddings didn't).

AI Code Review at Scale

> *Claude Code in GitHub Actions*

Automated PR Review

```yaml
# .github/workflows/ai-review.yml
name: AI Code Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Claude Code Review
        run: |
          claude -p "Review this PR diff.
          Categorize findings as:
          - MUST FIX: bugs, security issues
          - SHOULD CONSIDER: performance, readability
          - MINOR: style, naming suggestions
          Be specific. Reference file:line."
```

Structured Review Output

```
## MUST FIX (2 issues)
1. **SQL Injection** - `src/api/users.ts:42`
   Raw string interpolation in query.
   Use parameterized queries instead.
2. **Missing auth check** - `src/api/admin.ts:15`
   Admin endpoint has no authentication middleware.

## SHOULD CONSIDER (1 issue)
1. **N+1 query** - `src/api/posts.ts:28`
   Fetching author inside a loop. Use JOIN.

## MINOR (1 issue)
1. **Naming** - `src/utils/helpers.ts:5`
   `processData()` too generic ->
`transformUserResponse()`
```

The human reviewer still makes the final call. AI review is a first pass, not a replacement.

The Future of AI Engineering

> *Where this is all heading*

The Trajectory

You are learning the middle column right now.

Emerging Trends

Computer Use Agents -- AI that uses a browser, clicks buttons, fills forms. Testing: "Verify the signup flow works end-to-end."

Multi-Modal Agents -- Screenshots + code + logs + docs in one context. "Here's the Figma design, here's the current page, make them match."

Agent Marketplaces -- Pre-built agents for specific tasks (security audit, migration, i18n). Plug into your CI/CD pipeline.

AI as Infrastructure -- AI becomes a standard pipeline stage, not a tool you invoke manually.

Career Implications

Old World	New World
Write code from scratch	Direct AI to write code, review output
Memorize syntax	Understand architecture and systems
Individual productivity	Orchestrate AI agents and human teams
Debug by reading code	Debug by asking the right questions

Your advantage: You understand both engineering fundamentals AND AI tooling. That combination is rare and valuable.

Demo Preparation

> *Presenting your P3 to the class*

Demo Day Structure

Each team gets 8-10 minutes.

Section	Time	Focus
Problem Statement	1 min	What problem? Who cares?
Architecture Overview	2 min	Mermaid diagram, tech stack
Live Demo	3 min	Core user flow, working app
AI Workflow Showcase	2 min	How AI helped build this
Q&A	2 min	Peer questions

Problem Statement & Architecture

Problem Statement (1 min) -- answer three questions:

1. **What problem does this solve?** Be specific: "Students can't find study partners for specific courses" -- not "We built a social app"
2. **Who has this problem?** Your target user, from Mom Test research
3. **Why does your solution matter?**

Architecture (2 min) -- show a Mermaid diagram:

```
graph LR
  A[React Frontend] --> B[Next.js API]
  B --> C[PostgreSQL]
  B --> D[Auth0]
  F[GitHub Actions] --> G[Vercel]
```

Highlight tech stack choices, key decisions, where AI agents fit.

Live Demo & AI Showcase

Live Demo (3 min):

1. **Script it.** Know exactly what you'll click, type, show
2. **Use real data.** Not "test test test"
3. **Show the core flow.** Login -> main feature -> result
4. **Have a backup.** Record a video in case WiFi fails
5. **Practice at least 3 times.**

AI Workflow Showcase (2 min):

- A complex feature where Claude Code wrote the implementation
- A bug where AI diagnosis saved hours
- Your CLAUDE.md and how it evolved
- Your CI/CD pipeline with AI review

Peer Evaluation Rubric

Criterion	Excellent (5)	Good (3-4)	Needs Work (1-2)
Clarity	Problem and solution immediately clear	Mostly clear, minor confusion	Hard to understand
Technical Depth	Architecture well-explained	Some depth, some hand-waving	Surface level only
Demo Quality	Smooth, real data, core flow	Minor hiccups, mostly smooth	Broken, unprepared
AI Workflow	Clear integration story, specific examples	Some AI usage shown	No AI workflow discussed
Q&A	Confident, knowledgeable	Reasonable answers	Unable to answer

Meta-Reflection

> *The most important exercise of the semester*

Why Reflect?

Learning without reflection is just experience.

Students who reflect on their learning retain information 23% better, transfer skills to new contexts more effectively, and develop stronger metacognitive abilities.

This is not a "soft" exercise. It's a learning multiplier.

Questions to Answer

About your coding abilities:

- What did you learn about your own strengths and weaknesses?
- How has your relationship with code changed?

About AI as a tool:

- When did AI help you most? When did it hinder you?
- How did your prompting evolve from Week 1 to Week 14?
- What would you never let AI do? What do you always let it do?

About software quality:

- How has your understanding of "good code" changed?
- What engineering practices will you carry forward?

Hindsight: If you restarted P1 today, what would you change? Which AI modality was most valuable for which tasks? What's the biggest mistake you made?

Course Synthesis

> *Connecting the threads across Weeks 10-14*

The Four SE Practices

Practice	W10	W11	W12	W13
Testing & CI/CD	Tool permissions	TDD, GitHub Actions	Hooks enforce CI	Agent evals
Code Review	CLAUDE.md standard	PR workflow via CC	Writer/reviewer agents	Multi-agent review
System Design	Agentic loop	Explore-Plan-Implement	Skills, hooks, MCP	6 age patter
Ethical AI Dev	Permission modes	Human review of AI	Hooks as guardrails	Agent safety

The Learning Arc

```
Weeks 1-3: Understanding AI
  "What are LLMs? How do prompts work?"

Weeks 4-5: Using AI (Claude Web)
  "I can build prototypes with AI!"

Weeks 6-8: Coding with AI (IDE)
  "AI is my pair programmer"

Weeks 10-12: Engineering with AI (Claude Code)
  "AI is part of my development infrastructure"

Weeks 13-14: Architecting with AI (Agents &
Production)
  "I can design AI-powered systems"
```

You went from user to architect in 14 weeks.

From Individual to Production

```
P1: Individual Mastery
  Solo project, Claude Web, basic engineering
  "Can I build something useful with AI?"

P2: Team Development
  Pair project, IDE AI, Agile/Scrum, code review
  "Can I build with a partner and maintain quality?"

P3: Production Responsibility
  Team project, Claude Code, agents, CI/CD,
deployment
  "Can I ship a production-quality application?"
```

Each project raised the stakes and the standards.

What You Take With You

Technical skills: Prompt engineering, context engineering (CLAUDE.md, rules files), TDD with AI, CI/CD with AI review, agent architecture patterns.

Engineering judgment: When to trust AI output and when to verify, how to review AI-generated code, when to use which modality, how to manage costs and quality tradeoffs.

These are career-defining skills, not just course material.

Deliverables

> *What's due and what's next*

This Week

Deliverable	Due	Details
Weekly Quiz 14	Tuesday, Week 14 at 2:59 PM PT	Production, cost optimization, emerging trends
P3 Sprint 4	End of week	Deploy and polish. Final sprint!

P3 Final Sprint Checklist:

- ☐ App deployed to production (Vercel or similar)
- ☐ CI/CD pipeline passing all stages
- ☐ Environment variables properly configured
- ☐ Error monitoring set up (Sentry or equivalent)
- ☐ README with setup instructions and live URL
- ☐ Demo video recorded as backup
- ☐ Architecture diagram (Mermaid) in repo

Next Week: Week 15

No lecture.

Item	Details
Project 3 Due	Final submission with deployed URL
Demo Videos	Optional async demo submissions

This is your last chance to polish before submission.

Resources

> *Readings and references for this week*

Required Readings

Resource	Description	URL
Prompt Caching	How caching works, when to use it	platform.claude.com/docs/en/bui with-claude/prompt-caching
Define Success Criteria	Building measurable success metrics	platform.claude.com/docs/en/tes and-evaluate/define-success
OWASP Top 10	Security best practices for web apps	owasp.org/www-project-top-ten/
GitHub Actions Docs	CI/CD pipeline reference	docs.github.com/en/actions
Vercel Deployment Docs	Production deployment	vercel.com/docs

Recommended Readings

Resource	Description	URL
OpenSSF AI Code Security Guide	Security-focused best practices	best.openssf.org/Security-Foc AI-Code-Assistant-Instructions
Reduce Hallucinations	Verification strategies and guardrails	platform.claude.com/docs/en/t evaluate/strengthen-guardrails hallucinations
When to Trust AI-Generated Code	Guidelines for code trust	graphite.com/guides/when-to-
GitHub Copilot Research	Does Copilot improve code quality?	github.blog/news-insights/rese github-copilot-improve-code-q
Cybersecurity Risks of AI Code	Georgetown CSET report	cset.georgetown.edu/publicati risks-of-ai-generated-code/

Thank You

Last lecture of the semester.

You started this course wondering if "vibe coding" was real engineering.

Now you know: **it is, when you do it right.**

Good luck with your demos and final submissions.

Build something you're proud of.

Speaker notes